

4.Git-HOL

Installing Git

```
KIIT@BT-22051573 MINGW64 ~  
$ git --version  
git version 2.50.1.windows.1
```

Configuring the username and user-email

```
KIIT@BT-22051573 MINGW64 ~  
$ git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=openssl  
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt  
core.autocrlf=true  
core.fscache=true  
core.symlinks=true  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
gui.recentrepo=C:/Users/KIIT/https<U+F03A>github.com/theforage/forage-jpmc-swe-task-1  
user.name=AmritaKumari17  
user.email=amritakumari170105@gmail.com  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
core.editor=notepad++
```

Making new folder and initializing git

```
KIIT@BT-22051573 MINGW64 ~  
$ mkdir GitMergeConflictDemo  
cd GitMergeConflictDemo  
git init  
Initialized empty Git repository in C:/Users/KIIT/GitMergeConflictDemo/.git/
```

1. Verify if master is in clean state.

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git status
On branch master

No commits yet

nothing to commit (create/copy files and use "git add" to track)
```

2. Create a branch “GitWork”. Add a file “hello.xml”.

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'
```

3. Update the content of “hello.xml” and observe the status

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (GitWork)
$ echo "<message>Hello from GitWork branch</message>" > hello.xml
git add hello.xml
git status
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it
On branch GitWork

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   hello.xml
```

4. Commit the changes to reflect in the branch

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (GitWork)
$ git commit -m "Added hello.xml in GitWork branch"
[GitWork (root-commit) 13b6a25] Added hello.xml in GitWork branch
1 file changed, 1 insertion(+)
create mode 100644 hello.xml
```

5. Switch to master.

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (GitWork)
$ git checkout -b master
Switched to a new branch 'master'
```

6. Add a file “hello.xml” to the master and add some different content than previous and 7. Commit the changes to the master

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ echo "<message>Hello from master branch</message>" > hello.xml
git add hello.xml
git commit -m "Added hello.xml in master branch"
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it
[master 8e358c1] Added hello.xml in master branch
1 file changed, 1 insertion(+), 1 deletion(-)
```

8. Observe the log by executing “git log –oneline –graph –decorate –all”

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git log --oneline --graph --decorate --all
* 8e358c1 (HEAD -> master) Added hello.xml in master branch
* 13b6a25 (GitWork) Added hello.xml in GitWork branch
```

9. Check the differences with Git diff tool

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git diff master GitWork
diff --git a/hello.xml b/hello.xml
index 82868a7..0bf4c19 100644
--- a/hello.xml
+++ b/hello.xml
@@ -1,1 @@
- <message>Hello from master branch</message>
+ <message>Hello from GitWork branch</message>
```

10. For better visualization, use P4Merge tool to list out all the differences between master and branch

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git config --global merge.tool p4merge
git config --global diff.tool p4merge
```

11. Merge the bran to the master

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git merge GitWork
Already up to date.
```

12. Observe the git mark up.

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ cat hello.xml
<message>Hello from master branch</message>
```

13. Use 3-way merge tool to resolve the conflict

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git mergetool
No files need merging
```

14. Commit the changes to the master, once done with conflict

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git add hello.xml
git commit -m "Resolved merge conflict in hello.xml"
On branch master
nothing to commit, working tree clean
```

15. Observe the git status and add backup file to the .gitignore file and 16. Commit the changes to the .gitignore

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ echo "*.orig" >> .gitignore
git add .gitignore
git commit -m "Added *.orig to .gitignore"
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
[master 1d7a73c] Added *.orig to .gitignore
1 file changed, 1 insertion(+)
create mode 100644 .gitignore
```

17. List out all the available branches

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git branch
  GitWork
* master
```


18. Delete the branch, which merge to master.

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git branch -d GitWork
Deleted branch GitWork (was 13b6a25).
```

19. Observe the log by executing “git log --oneline --graph --decorate”

```
KIIT@BT-22051573 MINGW64 ~/GitMergeConflictDemo (master)
$ git log --oneline --graph --decorate
* 1d7a73c (HEAD -> master) Added *.orig to .gitignore
* 8e358c1 Added hello.xml in master branch
* 13b6a25 Added hello.xml in GitWork branch
```